Protecting the Right Weights on a Noisy Hybrid SLC/MLC ReRAM Chip

Author One
Department
Institution
City, Country
email@example.com

Author Two
Department
Institution
City, Country
email@example.com

***Abstract*—Analog resistive-RAM (ReRAM) processing-in-memory computes matrix–vector products inside the memory array and removes most of the data movement that dominates transformer inference. Its weakness is that a cell cannot be programmed exactly. A single-level cell (SLC) stores one bit and tolerates this error, while a multi-level cell (MLC) stores two bits and is denser but far less robust. A hybrid chip keeps the important weights in SLC and the rest in MLC, so the central question is which weights to protect. We study this question for DistilBERT and BERT-Base on SST-2 with a cell-level crossbar simulator that models 8-bit weights, differential pairs, bit-slicing, programming noise, stuck-at faults, read noise, a fixed ADC and IR drop. After a singular value decomposition (SVD) of every weight matrix, we compare six placements of the SVD components: all-SLC, all-MLC, random, largest singular value (magnitude), a HyFlexPIM-style loss gradient, and a Fisher-information estimate of the damage each component would suffer in MLC. Storing everything in MLC fails at realistic noise, and BERT-Base falls from about 90% to near chance. Protecting a small share of components recovers almost all of the loss, but only if the right components are chosen. Ranking by singular value matches the best placement we could construct, whereas the gradient placement loses 3–12 accuracy points at high noise. Stuck-at cells are the one failure that placement does not fix: all-MLC fails at 0.1% stuck cells and all-SLC at 1%.**

***Index Terms*—processing-in-memory, ReRAM, multi-level cell, transformer, singular value decomposition, fault tolerance**

I. Introduction

Transformer models [2] are expensive to run, mainly because billions of weights must be moved between memory and the processor. Processing-in-memory (PIM) removes this movement by computing where the weights are stored. Analog ReRAM crossbars are an efficient form of PIM, but a cell cannot be programmed exactly, so every stored weight carries error. Dense multi-level cells (MLC) suffer most from this error, and reliable single-level cells (SLC) are costly. A hybrid chip therefore keeps only the most important weights in SLC, and the open question is which weights these are.

This paper studies that question for SVD-factorised transformers on a simulated hybrid SLC/MLC ReRAM chip. Section II gives the background, including the HyFlexPIM method [1] and our contributions. The remaining sections will describe the simulator, the experiments and the results.

II. Background

A. Analog ReRAM Crossbars

A ReRAM crossbar computes a matrix–vector product inside memory. Each weight is stored as the conductance of resistive cells. A voltage proportional to each input is applied to a row, and the currents on each column add up to the output. This avoids most of the data movement of conventional hardware, and it computes a whole column in one step.

The cost is accuracy. A cell cannot be programmed to an exact conductance, and cells can also be stuck at a fixed conductance. A single-level cell (SLC) stores one bit in two well-separated levels and tolerates considerable error. A multi-level cell (MLC) stores two bits in four closely spaced levels. It needs fewer cells per weight, which gives higher density, throughput and energy efficiency, but the same absolute conductance error is a larger error in the stored value. Transformers are sensitive to such error, and running BERT entirely on two-bit cells can cost tens of accuracy points [1].

B. HyFlexPIM

HyFlexPIM [1] is a hybrid SLC/MLC mixed-signal PIM accelerator. A single analog PIM module can switch between SLC and MLC operation with under 1% area and energy overhead, so critical weights can be stored in SLC and the remaining weights in the denser MLC. Brute-force mapping on such a chip gives little benefit, because too few weights are error-tolerant enough for MLC. HyFlexPIM therefore adds an algorithmic step called *gradient redistribution*:

1) Factorise each weight matrix with singular value decomposition (SVD) into components ordered by singular value.
2) Truncate the smallest components to remove redundancy.
3) Fine-tune the truncated model, which concentrates the loss gradient in a small subset of components.
4) Rank the components by the absolute gradient of the loss with respect to their singular value.
5) Store the top $k\%$ in SLC and the remaining components in MLC.

All of these steps run in software before deployment, and the hardware only receives the final truncated matrices. HyFlex-

PIM models MLC error by multiplying each weight by $(1+\eta)$ with Gaussian η, with the variance chosen to match a bit error rate of about 4.04% measured on fabricated MLC RRAM. It reports that protecting 5–30% of components in SLC keeps most GLUE tasks within 1% of the noise-free baseline, and that gradient-based selection outperforms magnitude- and rank-based selection on MRPC and CoLA with BERT-Base.

C. Open Questions and Our Contributions

The evidence for gradient-based selection covers a small number of tasks and a single noise model, and the model has no stuck-at cells. It is unclear whether the rule holds under other error models, whether a cheaper rule performs as well, and how stuck-at cells affect each placement. Our contributions address these points:

- **A crossbar simulator.** A cell-level PyTorch model of SVD-factorised transformers on SLC and MLC crossbars, covering weight quantisation, differential pairs, bit-slicing, programming noise, stuck-at faults, read noise, a fixed ADC and IR drop. All placements are evaluated on the same simulated chips.
- **Criticality-aware component placement.** A comparison of six policies for choosing which SVD components are stored in SLC: all-SLC, all-MLC, random, singular-value magnitude, the HyFlexPIM gradient, and a Fisher-information estimate of the damage each component would suffer in MLC.
- **Variation with SLC share.** How the accuracy of each placement policy changes as the share of components stored in SLC increases.
- **Fault analysis.** An evaluation of how programming noise and stuck-at cells, which HyFlexPIM does not model, affect each placement policy. We study DistilBERT and BERT-Base on the SST-2 task of GLUE.

III. Methodology

Our pipeline has three stages: SVD factorisation of the transformer, a cell-level crossbar simulation, and the placement of the SVD components on SLC and MLC cells. This section describes the first stage.

A. SVD of the Transformer

1) Models and dataset: We use two encoder transformers, DistilBERT [3] and BERT-Base [2], and the SST-2 sentiment task of GLUE [4]. SST-2 is a binary classification task. It has 67,349 training sentences and 872 validation sentences, and all accuracies in this paper are measured on the validation set. Both models start from public checkpoints that are already fine-tuned on SST-2 (`distilbert-base-uncased-finetuned-sst-2-english` and `textattack/bert-base-uncased-SST-2`). Sentences are truncated to 128 tokens.

TABLE I
SST-2 validation accuracy (%) of the SVD-factorised models after fine-tuning.

Model	SVD + fine-tune
DistilBERT	89.11
BERT-Base	90.14

2) Factorisation: Following HyFlexPIM, we factorise every linear layer of the encoder blocks (the query, key, value and output projections and the two feed-forward layers) as

$$W \approx U\,\mathrm{diag}(\sigma)\,V^{\top}, \qquad W \in \mathbb{R}^{m\times n}, \tag{1}$$

which gives 36 factorised layers for DistilBERT and 72 for BERT-Base. We keep the $k = \lfloor mn/(m+n) \rfloor$ largest singular values (for example $k = 384$ for a 768×768 layer), the largest rank for which the two factors U and $\sigma V^{\top}$ together hold no more parameters than W. On the chip each layer then becomes two crossbars, $x \to [\sigma V^{\top}] \to [U] \to y$, and component r consists of row r of $\sigma V^{\top}$ together with column r of U. This gives 16,584 components for DistilBERT and 33,168 for BERT-Base.

3) Recovery fine-tuning: Truncating to rank k removes information, so we fine-tune the factorised model on the SST-2 training set to recover the lost accuracy. We use AdamW with a learning rate of 2×10^{-5} decayed linearly to zero, batch size 32, 8 epochs, mixed-precision (fp16) autocast and seed 0. Table I shows the validation accuracy of the SVD-factorised models after fine-tuning.

All later experiments use these fine-tuned factorised models as their noise-free reference, so the accuracies in the tables that follow should be read against 89.11% (DistilBERT) and 90.14% (BERT-Base).

References

[1] C. E. Song, P. Bhatnagar, Z. Xia, N. S. Kim, T. S. Rosing, and M. Kang, "Hybrid SLC-MLC RRAM mixed-signal processing-in-memory architecture for transformer acceleration via gradient redistribution," in *Proc. 52nd Annu. Int. Symp. Computer Architecture (ISCA)*, Tokyo, Japan, Jun. 2025, pp. 1155–1170.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[4] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. ICLR*, 2019.